

ASP.NET Core Web API Project File

Back to: [ASP.NET Core Web API Tutorials](#)

ASP.NET Core Web API Project File

In ASP.NET Core, the project file (with a .csproj extension) plays a crucial role in defining how your application is built, the packages it references, and the frameworks it targets. Compared to earlier .NET Framework versions, the .csproj format in .NET Core is streamlined, cleaner, and far more developer-friendly.

What is a Project File in ASP.NET Core Web API?

In ASP.NET Core, the project file (.csproj for C# or .vbproj for VB.NET) serves as the backbone of the application's configuration. It is an XML-based file that defines how the application is built, which frameworks it targets, and what external dependencies or NuGet packages it requires. Unlike older .NET Framework project files, the .csproj file in ASP.NET Core is simplified, cleaner, and more developer-friendly.

Key improvements in ASP.NET Core project files include:

- **Readability** – The XML is short, minimal, and easier to understand.
- **Cross-Platform Support** – A single format works across Windows, macOS, and Linux.
- **On-the-fly Editing** – The file can be edited directly in Visual Studio without needing to be unloaded.
- **Modularity** – Packages are added only when needed, keeping the project lightweight.
- **CLI Compatibility** – Works seamlessly with dotnet CLI commands for build, run, and publish.

How to Edit Project File in Previous Versions of ASP.NET?

In earlier versions of the ASP.NET Framework, project files were much more complex, and editing them required multiple steps. Developers had to unload the project from Visual Studio before making any changes, which made the process time-consuming and error-prone.

The typical steps were:

- **Unload Project** – Right-click the project in Solution Explorer and select 'Unload Project'.
- **Open File Manually** – Edit the .csproj file in Notepad or another XML editor.
- **Save Changes** – Manually save the modifications.
- **Reload Project** – Reload the project into Visual Studio to apply the updates.

Drawbacks of this old approach included:

- **Time Consumption** – Every small edit required four separate steps.
- **Error-Prone** – A single mistake in the XML could corrupt the project configuration.
- **Workflow Disruption** – Developers had to interrupt coding frequently just to tweak project settings.

How to Edit a Project File in ASP.NET Core Web API?

With ASP.NET Core, Microsoft redesigned the system to make editing project files far more convenient. Now, developers can open and modify the .csproj file directly within Visual Studio without needing to unload the project.

The simplified process is:

- **Right-Click → Edit Project File** – The project file opens directly in the Visual Studio editor.
- **No Unloading Required** – You can keep working while editing the project file.
- **Immediate Updates** – Any saved changes are instantly reflected in the project.

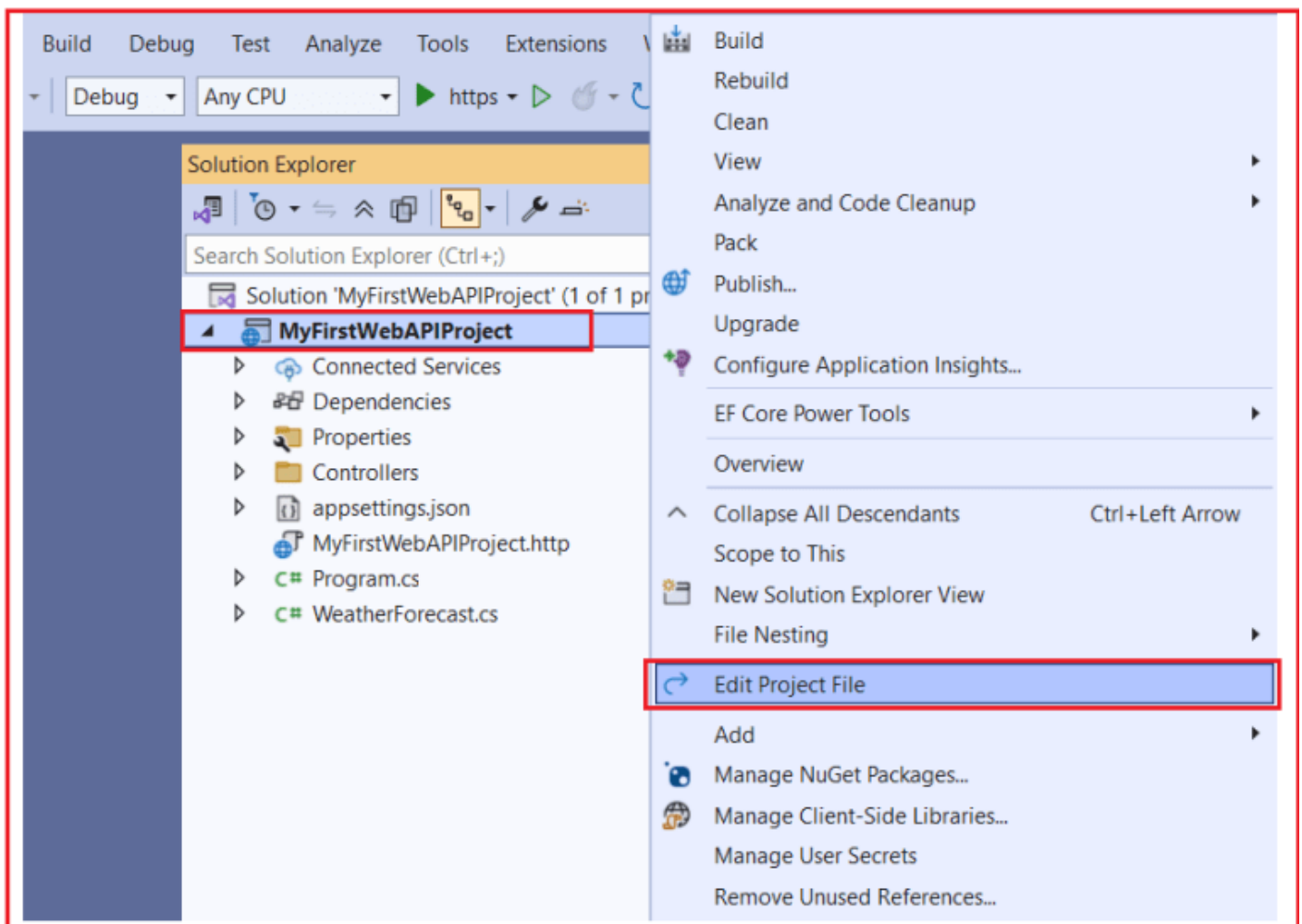
Benefits of this new approach:

- **Faster Workflow** – No interruption to development activities.
- **Greater Productivity** – Developers can quickly add dependencies, change frameworks, or enable features.
- **Error Reduction** – Built-in Visual Studio support helps avoid mistakes while editing XML.

Default Structure of an ASP.NET Core Web API Project File

When you create an ASP.NET Core application, Visual Studio automatically generates a project file with the .csproj extension. This file is not just a list of included source code; it is an **XML-based configuration file** that defines the project's behavior. It tells the compiler **how to build the application, which framework to target, and which dependencies, packages, and tools** are required.

To view or edit the project file in Visual Studio, you can **right-click the project in Solution Explorer → Edit Project File as shown in the image below**. Unlike older versions of .NET Framework, the project file opens directly in the Visual Studio editor, allowing you to modify it without unloading or reloading the project. This makes managing settings and dependencies much faster and more convenient.



Once you click on the **Edit Project File**, the Project file will open in the Visual Studio editor as shown in the image below.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Key Components of the Project File in ASP.NET Core:

An ASP.NET Core project file contains multiple sections that define how the application behaves. Each section has a specific purpose and contributes to the build and runtime process:

<Project Sdk="Microsoft.NET.Sdk.Web">

This line defines the **Software Development Kit (SDK)** that the project will use. In ASP.NET Core applications, the **Microsoft.NET.Sdk.Web** SDK is the most common choice, as it sets up the project with everything required to build and run modern web applications.

- Declares that the project is an **ASP.NET Core Web Application**.
- Provides support for **Web APIs, MVC applications, and Razor Pages**.
- Automatically includes the necessary **tools, libraries, and templates**.
- If you were building a console app, you would see `Microsoft.NET.Sdk` instead.

<PropertyGroup> Section

The `<PropertyGroup>` element stores project-level properties that affect compilation, build settings, and runtime behavior. Multiple `PropertyGroup` blocks can exist to support different build configurations (e.g., Debug vs. Release).

Common elements inside PropertyGroup:

- **Target Framework:** `<TargetFramework>net8.0</TargetFramework>`
 - Indicates that the project is built against **.NET 8.0**.
 - Ensures access to the latest APIs, libraries, and optimizations provided by .NET 8.
 - Guarantees compatibility with the specified runtime.
- **Nullable Reference Types:** `<Nullable>enable</Nullable>`
 - Introduced in **C# 8.0** to help developers avoid null reference exceptions.
 - The compiler generates warnings when a variable may potentially be null.
 - Makes applications more **robust, reliable, and safe**.
- **Implicit Usings:** `<ImplicitUsings>enable</ImplicitUsings>`
 - Introduced in **.NET 6**.
 - Automatically imports common namespaces like `System`, `System.Linq`, `Microsoft.AspNetCore.Http`.
 - Reduces the need to repeat using statements in every file.
 - Keeps source code **cleaner and more readable**.

<ItemGroup> Section

The `<ItemGroup>` element is used to define collections of items such as dependencies, references, or even files. In the context of package management, it is primarily used to specify **NuGet package references**.

PackageReference: `<PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />`

- Declares that the project depends on the **Swashbuckle.AspNetCore** NuGet package.
- This package provides **Swagger/OpenAPI documentation support** for ASP.NET Core APIs.
- Once added, the package will appear under **Dependencies** → **Packages** in Visual Studio.
- If the package is removed, the entry will disappear from both the .csproj file and Solution Explorer.

Other uses of ItemGroup:

- Managing **project references** when linking multiple projects together.
- Including **folders or files** (e.g., Models, Views) in the project structure.

In summary:

- The **Project SDK** tells Visual Studio what kind of project you are building.
- The **PropertyGroup** defines how the project should behave during build and runtime.
- The **ItemGroup** manages dependencies, project references, and other included items.

Adding Packages in ASP.NET Core:

ASP.NET Core follows a **modular approach**, which means that, unlike the old .NET Framework, it does not come preloaded with a large number of assemblies. By default, only the most essential components required to run the application are included. Everything else—such as JSON processing libraries, Swagger for API documentation, or authentication providers—must be explicitly added as packages. This approach keeps applications **lightweight, fast, and customizable**.

When you add a package to your ASP.NET Core project, two things happen simultaneously:

- The package gets listed under the **Dependencies** → **Packages** section inside Visual Studio.
- A new <PackageReference> entry is added to the .csproj project file, ensuring that the package is restored and included whenever the project is built.

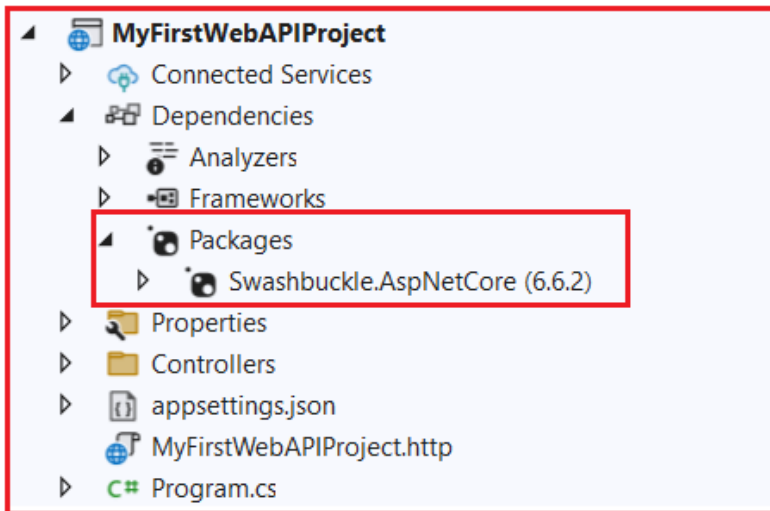
How to Add a Package Using NuGet Package Manager

Visual Studio provides a built-in interface called **NuGet Package Manager**, which makes it easy to add external libraries.

Steps to add a package:

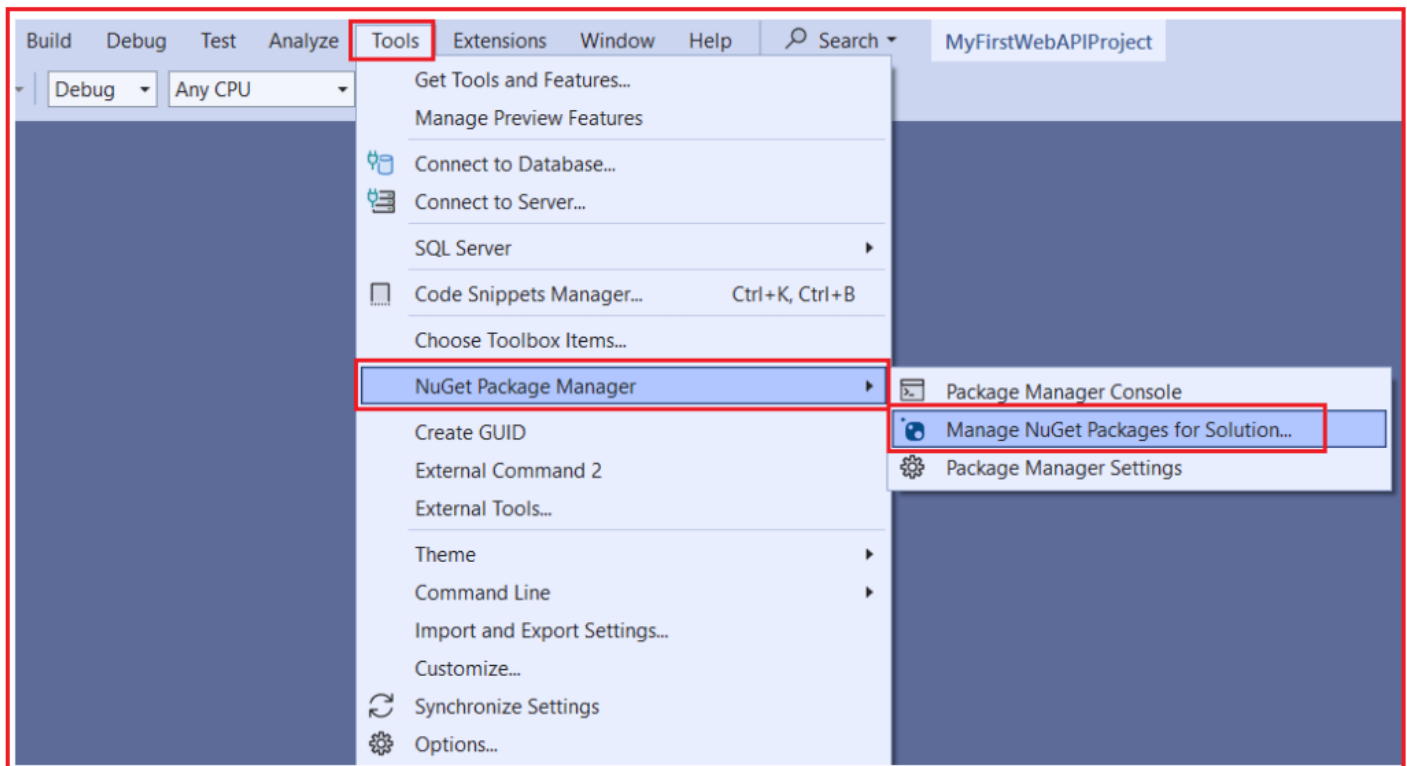
- Go to **Tools** → **NuGet Package Manager** → **Manage NuGet Packages for Solution....**
- In the **Browse tab**, search for the package name (e.g., Newtonsoft.Json).
- Select the desired version and click **Install**.
- Accept any license agreements, and the package will be added to your project.

Whenever we add a new package to our application, that package reference is also added to the application project file. Please review the Dependencies section of our project. Whenever we add any packages, those packages and their dependency packages will be stored here.

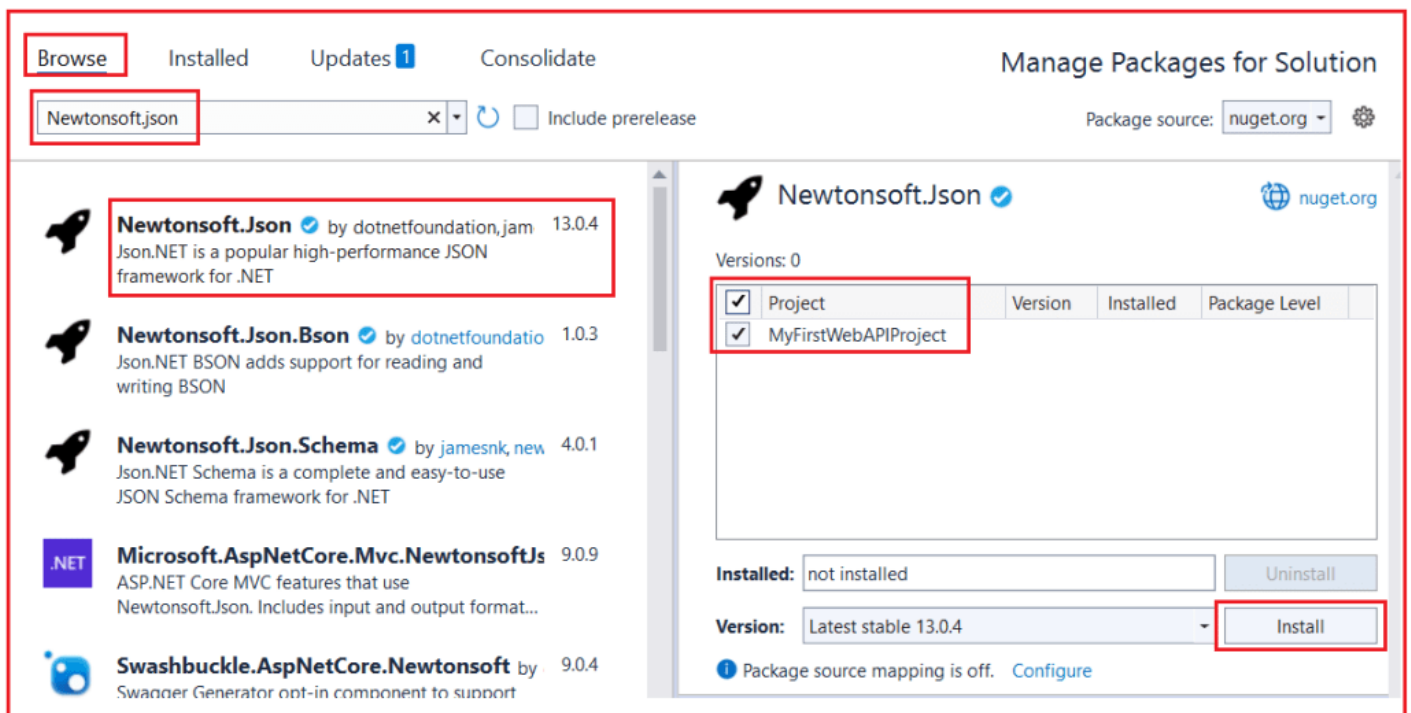


Example: Adding the Newtonsoft.Json Package

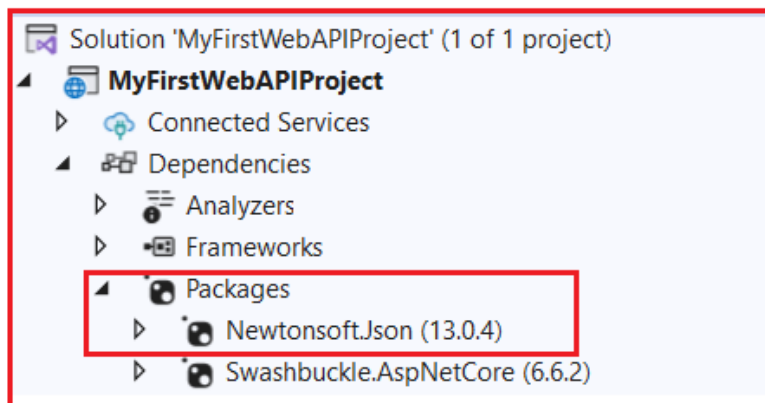
Let us first add a package from the NuGet Package Manager solution and see what happens. Go to **tools => NuGet Package Manager => Manage NuGet Packages for Solution...** option from the context menu as shown in the image below.



Now, let us add the **Newtonsoft.json** package. So, select the browse tab and then search for Newtonsoft and install it as shown in the image below.



Once the package is installed successfully, you will see that it adds a reference to the **Dependencies/Packages** section, as shown in the image below.



At the same time, it will also add that package reference in the application project file, as shown in the image below.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

What Happens When You Remove a Package?

If you later decide that the package is no longer required. Removing it via **NuGet Package Manager** or by deleting the `<PackageReference>` entry from `.csproj` will also remove it from both:

- The **Dependencies/Packages** node in Visual Studio.
- The project file itself.

This automatic cleanup helps prevent unnecessary bloat in the application and keeps dependencies under control.

Creating a Folder:

In ASP.NET Core, when you organize your project by creating new folders (for example, a folder named **Models** at the project root), Visual Studio automatically tracks these changes. The **project file (.csproj)** is designed to recognize and include folders, files, and resources that are part of the project structure.

When you create a new folder, such as **Models**:

- A reference to that folder is automatically reflected in the **project file (.csproj)**.
- This ensures that any **classes, configuration files, or resources** added to that folder are included in the build process.
- Developers typically use folders such as **Models, Controllers, Views, Services, and Repositories** to maintain a clean and modular architecture.

For example:

- A **Models** folder may contain entity classes (such as `Customer.cs` and `Product.cs`) that represent the data structures used in the application.

- A **Controllers** folder will hold controller classes that handle HTTP requests.
- A **Views** folder (in MVC projects) stores Razor view files (.cshtml).

Example of Folder Reference in .csproj

If you create a folder called **Models**, you may see something like this added:

```
<ItemGroup>
  <Folder Include="Models\" />
</ItemGroup>
```

This line tells the project system:

- There is a **Models** folder in the project root.
- Any files (e.g., Customer.cs, Product.cs) placed in that folder are considered part of the project.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

  <ItemGroup>
    <Folder Include="Models\" />
  </ItemGroup>

</Project>
```

Why Does the Project File Track Folders?

- **Automatic Organization** → Keeps the project file synchronized with the actual folder structure of your application.
- **Automatic tracking** → Visual Studio ensures that newly created folders and files are linked to the project.
- **Build Inclusion** → Ensures files inside the folder are compiled or treated correctly based on their type.
- **Project Portability** → If the project is shared with another developer, the folder structure and references remain intact.
- **Cross-platform consistency** → Whether opened in Visual Studio, Visual Studio Code, or via CLI, the structure is preserved.

When you create a folder in an ASP.NET Core project (like **Models**), Visual Studio automatically updates the project file to recognize it. This integration helps keep the folder structure organized, ensures files are compiled correctly, and makes the project easier to maintain and share across teams.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information